

## Topic 5: Large Language Models

### 5.1 What Makes a Language Model 'Large'?

- Scale: LLMs have billions to hundreds of billions of parameters (learned weights)
- GPT-3 (2020): 175 billion parameters, trained on ~570 GB of text
- Scale unlocks emergent abilities that smaller models do not exhibit: multi-step reasoning, in-context learning, code generation
- Pre-training: the model learns to predict the next token on a massive, diverse corpus of internet text, books, and code
- Fine-tuning + RLHF: a pre-trained model is then shaped into a helpful assistant via supervised examples and human feedback

### 5.2 Major LLM Families

- OpenAI -- GPT-4o, GPT-4 Turbo: strongest general reasoning; powers ChatGPT and the OpenAI API ([openai.com/api](https://openai.com/api))
- Anthropic -- Claude 3.5 Sonnet, Claude 3 Opus: strong writing and instruction-following; 200k token context ([claude.ai](https://claude.ai))
- Google -- Gemini 1.5 Pro / Flash: natively multimodal; integrated into Google Workspace and Search ([gemini.google.com](https://gemini.google.com))
- Meta -- Llama 3 (8B, 70B, 405B): open weights, free to download and self-host ([llama.meta.com](https://llama.meta.com))
- Mistral AI -- Mistral 7B, Mixtral 8x7B: efficient open-source European models ([mistral.ai](https://mistral.ai))
- Explore open models: <https://huggingface.co/models>

### 5.3 Tokenisation

- A tokeniser splits text into tokens before the model processes it (not always full words)
- Example: 'unbelievable' might become ['un', 'believ', 'able'] -- 3 tokens
- Token counts matter: pricing and context limits are measured in tokens, not words
- Rough rule: 1 token ≈ 0.75 English words; 1,000 tokens ≈ 750 words
- Different models use different tokenisers -- the same text produces different token counts across models

```
# Count tokens with OpenAI's tiktoken library:
import tiktoken
enc = tiktoken.encoding_for_model('gpt-4o')
tokens = enc.encode('The quick brown fox jumps over the lazy dog.')
print(len(tokens)) # -> 10 tokens
```

## 5.4 In-Context Learning

- In-context learning: the model adapts its behaviour based on examples provided in the prompt -- no retraining needed
  - Zero-shot: give the task description only -- the model attempts it with no examples
  - Few-shot: provide 2--5 examples of input/output pairs before the actual task
  - Chain-of-thought: instruct the model to show reasoning steps: 'Think step by step'
  - The model does not update its weights during inference -- it reads the context and responds accordingly
-